

实验 2：预测鲍鱼年龄与银行房屋贷款判断建模及性能分析

一：实验目的

- 1、理解线性回归的基本概念；
- 2、掌握局部加权线性回归方法；
- 3、掌握算法程序实现流程；
- 4、掌握决策树算法程序设计；
- 5、理解决策树算法的一般应用。

二：实验环境

- 1、系统：windows 7、windows 10 或 windows 11
- 2、软件：PyCharm

三：实验内容

（一）鲍鱼年龄预测（1 页-11 页）

1、什么是回归？

回归的目的是预测数值型的目标值。最直接的办法是依据输入写出一个目标值的计算公式。假如你想预测自己汽车的功率，可能会这么计算：

汽车的功率 = $0.0015 * \text{年薪} - 0.99 * \text{收听公共广播的时间}$

这就是所谓的回归方程（regression equation），其中的 0.0015 和 -0.99 称为回归系数（regression weights），求这些回归系数的过程就是回归。一旦有了这些回归系数，再给定输入，做预测就非常容易了。具体的做法是用回归系数乘以输入值，再将结果全部加在一起，就得到了预测值。

说到回归，一般都是指线性回归（linear regression），所以本次实验里的回归和线性回归代表同一个意思。线性回归意味着可以将输入项分别乘以一些常量，再将结果加起来得到输出。需要说明的是，存在另一种成为非线性回归的回归模型，该模型不认同上面的做法，比如认为输出可能是输入的乘积。这样，上面的功率计算公式也可以写做：

汽车的功率 = $0.0015 * \text{年薪} / \text{收听公共广播的时间}$

这就是一个非线性回归的例子，本次实验对此不做深入讨论。

2、揭开回归的神秘面纱

（1）用线性回归找到最佳拟合直线

应该怎么从一大堆数据里求出回归方程呢？假定输入数据存放在矩阵 X 中，结果存放在向量 y 中：

$$X = \begin{bmatrix} x_{11} & x_{12} & x_{13} \\ x_{21} & x_{22} & x_{23} \\ x_{31} & x_{32} & x_{33} \end{bmatrix}, \quad y = \begin{bmatrix} y_1 \\ y_2 \\ y_3 \end{bmatrix}$$

而回归系数存放在向量 w 中：

$$w = \begin{bmatrix} \omega_1 \\ \omega_2 \\ \omega_3 \end{bmatrix}$$

那么对于给定的数据 x_1 ，即矩阵 X 的第一列数据，预测结果 u_1 将会通过如下公式给出：

$$u_1 = \begin{bmatrix} x_{12} \\ x_{21} \\ x_{31} \end{bmatrix}^T \times \begin{bmatrix} \omega_1 \\ \omega_2 \\ \omega_3 \end{bmatrix}$$

现在的问题是，手里有数据矩阵 X 和对应的标签向量 y ，怎样才能找到 w 呢？一个常用的方法就是找出使误差最小的 w 。这里的误差是指预测 u 值和真实 y 值之间的差值，使用该误差的简单累加将使得正差值和负差值相互抵消，所以我们采用平方误差。

平方误差和可以写做：

$$\sum_{i=1}^m (y_i - x_i^T w)^2$$

用矩阵表示还可以写做：

$$\begin{aligned} & \frac{\partial (y - Xw)^T (y - Xw)}{\partial w} \\ &= \frac{\partial (y^T y - y^T Xw - w^T X^T y + w^T X^T Xw)}{\partial w} \\ &= \frac{\partial y^T y}{\partial w} - \frac{\partial y^T Xw}{\partial w} - \frac{\partial w^T X^T y}{\partial w} + \frac{\partial w^T X^T Xw}{\partial w} \\ &= 0 - X^T y - \frac{\partial (w^T X^T y)^T}{\partial w} + 2X^T Xw \\ &= 0 - X^T y - \frac{\partial y^T Xw}{\partial w} + 2X^T Xw \\ &= 0 - X^T y - X^T y + 2X^T Xw \\ &= 2X^T (y - Xw) \end{aligned}$$

为啥能这么变化，记住一个前提：若 $\mathbf{x}$ 为向量，则默认 $\mathbf{x}$ 为列向量， $\mathbf{x}^T$ 为行向量。将上述提到的数据矩阵 $\mathbf{X}$ 和标签向量 $\mathbf{y}$ 带进去，就知道为何这么变化了。

在继续推导之前，我们要先明确一个目的：找到 $\mathbf{w}$ ，使平方误差和最小。因为我们认为平方误差和越小，说明线性回归拟合效果越好。

现在，我们用矩阵表示的平方误差和对 $\mathbf{w}$ 进行求导：

$$\begin{aligned}
 & \frac{\partial (y - X\omega)^T (y - X\omega)}{\partial \omega} \\
 &= \frac{\partial (y^T y - y^T X\omega - \omega^T X^T y + \omega^T X^T X\omega)}{\partial \omega} \\
 &= \frac{\partial y^T y}{\partial \omega} - \frac{\partial y^T X\omega}{\partial \omega} - \frac{\partial \omega^T X^T y}{\partial \omega} + \frac{\partial \omega^T X^T X\omega}{\partial \omega} \\
 &= 0 - y^T X - \frac{\partial (\omega^T X^T y)^T}{\partial \omega} + 2X^T X\omega \\
 &= 0 - y^T X - \frac{\partial y^T X\omega}{\partial \omega} + 2X^T X\omega \\
 &= 0 - X^T y - X^T y + 2X^T X\omega \\
 &= 2X^T (y - X\omega)
 \end{aligned}$$

令上述公式等于 0，得到：

$$\hat{\omega} = (X^T X)^{-1} X^T y$$

$\mathbf{w}$ 上方的小标记表示，这是当前可以估计出的 $\mathbf{w}$ 的最优解。从现有数据上估计出的 $\mathbf{w}$ 可能并不是数据中的真实 $\mathbf{w}$ 值，所以这里使用了一个"帽"符号来表示它仅是 $\mathbf{w}$ 的一个最佳估计。

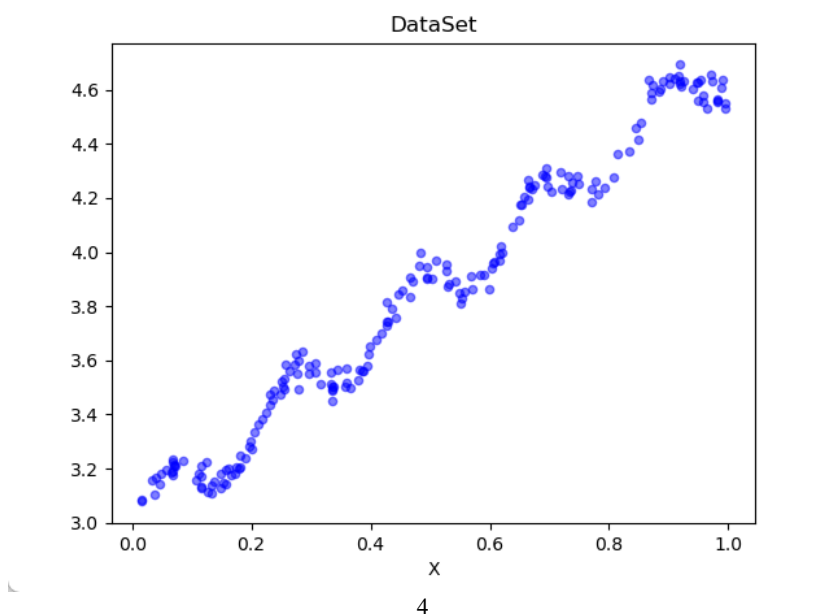
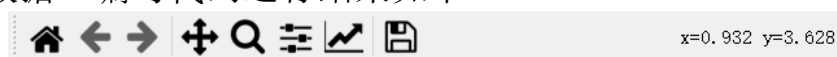
值得注意的是，上述公式中包含逆矩阵，也就是说，这个方程只在逆矩阵存在的时候使用，也即是这个矩阵是一个方阵，并且其行列式不为 0。

述的最佳 $\mathbf{w}$ 求解是统计学中的常见问题，除了矩阵方法外还有很多其他方法可以解决。通过调用 NumPy 库里的矩阵方法，我们可以仅使用几行代码就完成所需功能。该方法也称作 OLS，意思是“普通小二乘法”(ordinary least squares)。

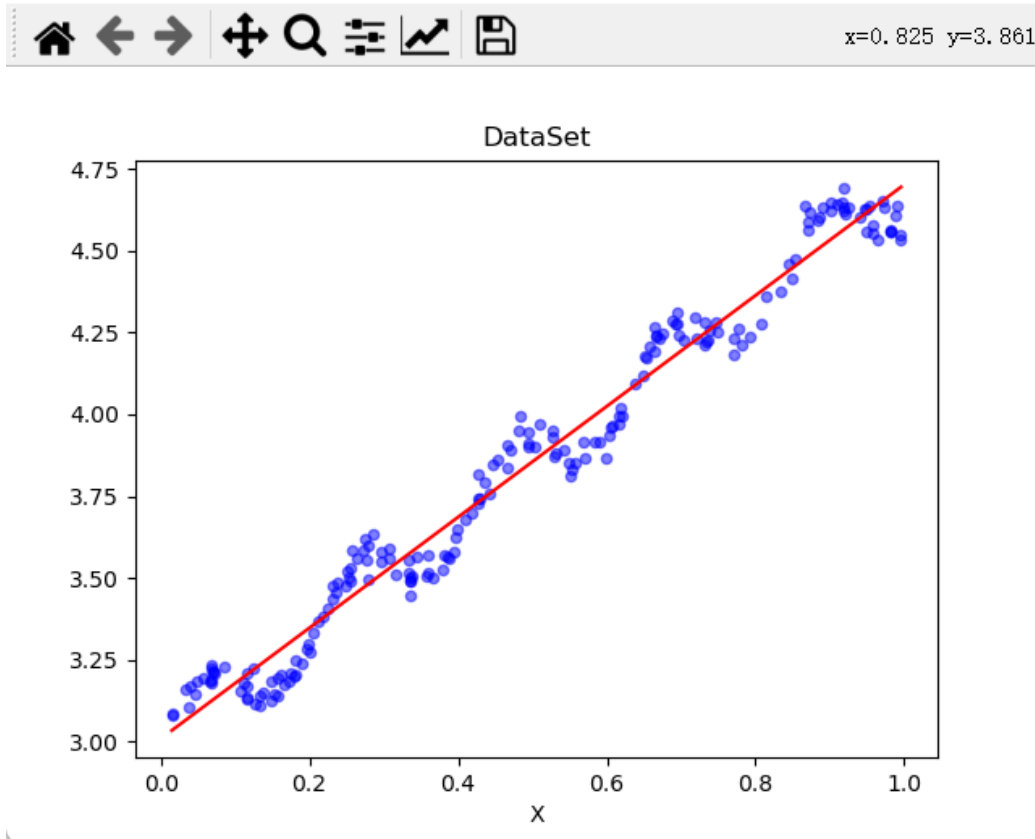
我们先看下数据集

1	1.000000	0.067732	3.176513
2	1.000000	0.427810	3.816464
3	1.000000	0.995731	4.550095
4	1.000000	0.738336	4.256571
5	1.000000	0.981083	4.560815
6	1.000000	0.526171	3.929515
7	1.000000	0.378887	3.526170
8	1.000000	0.033859	3.156393
9	1.000000	0.132791	3.110301
10	1.000000	0.138306	3.149813
11	1.000000	0.247809	3.476346
12	1.000000	0.648270	4.119688
13	1.000000	0.731209	4.282233
14	1.000000	0.236833	3.486582
15	1.000000	0.969788	4.655492
16	1.000000	0.607492	3.965162
17	1.000000	0.358622	3.514900
18	1.000000	0.147846	3.125947
19	1.000000	0.637820	4.094115
20	1.000000	0.230372	3.476039
21	1.000000	0.070237	3.210610
22	1.000000	0.067154	3.190612
23	1.000000	0.925577	4.631504
24	1.000000	0.717733	4.295890
25	1.000000	0.015371	3.085028
26	1.000000	0.335070	3.448080
27	1.000000	0.040486	3.167440
28	1.000000	0.212575	3.364266
29	1.000000	0.617218	3.993482
30	1.000000	0.541196	3.891471

第一列都为 1.0，即 x_0 。第二列为 x_1 ，即 x 轴数据。第三列为 x_2 ，即 y 轴数据。编写代码运行结果如下。



通过可视化数据，我们可以看到数据的分布情况。接下来，让我们根据上文中推导的回归系数计算方法，求出回归系数向量，并根据回归系数向量绘制回归曲线，编写代码运行结果如下。



如何判断拟合曲线的拟合效果的呢？当然，我们可以根据自己的经验进行观察，除此之外，我们还可以使用 `corrcoef` 方法，来比较预测值和真实值的相关性。编写代码运行结果如下。

```
D:\ProgramData\Anaconda3\python.exe
[[1.          0.98647356]
 [0.98647356 1.          ]]

进程已结束,退出代码0
```

可以看到，对角线上的数据是 1.0，因为 `yMat` 和自己的匹配是完美的，而 `YHat` 和 `yMat` 的相关系数为 0.98。

最佳拟合直线方法将数据视为直线进行建模，具有十分不错的表现。数据当中似乎还存在其他的潜在模式。那么如何才能利用这些模式呢？我们可以根据数据来局部调整预测，下面就会介绍这种方法。

3、局部加权线性回归

线性回归的一个问题是有可能出现欠拟合现象，因为它求的是具有小均方误

差的无偏估计。显而易见，如果模型欠拟合将不能取得好的预测效果。所以有些方法允许在估计中引入一些偏差，从而降低预测的均方误差。

其中的一个方法是局部加权线性回归（Locally Weighted Linear Regression, LWLR）。在该方法中，我们给待预测点附近的每个点赋予一定的权重。与 kNN 一样，这种算法每次预测均需要事先选取出对应的数据子集。该算法解出回归系数 $\hat{w}$ 的形式如下：

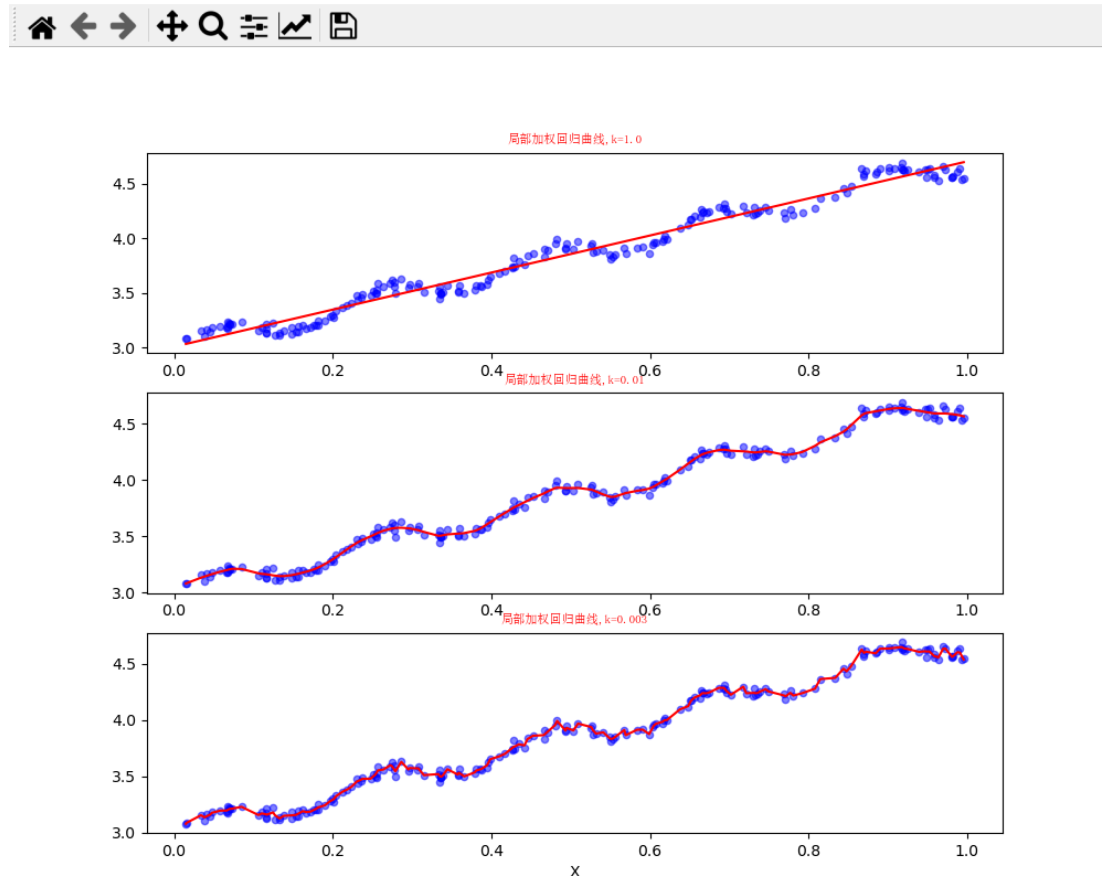
$$\hat{w} = (X^T W X)^{-1} X^T W y$$

其中 W 是一个矩阵，这个公式跟我们上面推导的公式的区别就在于 W ，它用来给每个点赋予权重。

LWLR 使用“核”（与支持向量机中的核类似）来对附近的点赋予更高的权重。核的类型可以自由选择，最常用的核就是高斯核，高斯核对应的权重如下：

$$w(i, i) = \exp\left(\frac{|x^{(i)} - x|^2}{-2k^2}\right)$$

这样我们就可以根据上述公式，编写局部加权线性回归，我们通过改变 k 的值，可以调节回归效果，编写代码运行结果如下。



可以看到，当 k 越小，拟合效果越好。但是当 k 过小，会出现过拟合的情况，例如 k 等于 0.003 的时候。

4、预测鲍鱼的年龄

接下来，我们将回归用于真实数据。在 `abalone.txt` 文件中记录了鲍鱼的年龄，这个数据来自 UCI 数据集合的数据。鲍鱼年龄可以从鲍鱼壳的层数推算得到。

数据集的数据如下：

1	1	0.455	0.365	0.095	0.514	0.2245	0.101	0.15	15
2	1	0.35	0.265	0.09	0.2255	0.0995	0.0485	0.07	7
3	-1	0.53	0.42	0.135	0.677	0.2565	0.1415	0.21	9
4	1	0.44	0.365	0.125	0.516	0.2155	0.114	0.155	10
5	0	0.33	0.255	0.08	0.205	0.0895	0.0395	0.055	7
6	0	0.425	0.3	0.095	0.3515	0.141	0.0775	0.12	8
7	-1	0.53	0.415	0.15	0.7775	0.237	0.1415	0.33	20
8	-1	0.545	0.425	0.125	0.768	0.294	0.1495	0.26	16
9	1	0.475	0.37	0.125	0.5095	0.2165	0.1125	0.165	9
10	-1	0.55	0.44	0.15	0.8945	0.3145	0.151	0.32	19
11	-1	0.525	0.38	0.14	0.6065	0.194	0.1475	0.21	14
12	1	0.43	0.35	0.11	0.406	0.1675	0.081	0.135	10
13	1	0.49	0.38	0.135	0.5415	0.2175	0.095	0.19	11
14	-1	0.535	0.405	0.145	0.6845	0.2725	0.171	0.205	10
15	-1	0.47	0.355	0.1	0.4755	0.1675	0.0805	0.185	10
16	1	0.5	0.4	0.13	0.6645	0.258	0.133	0.24	12
17	0	0.355	0.28	0.085	0.2905	0.095	0.0395	0.115	7
18	-1	0.44	0.34	0.1	0.451	0.188	0.087	0.13	10
19	1	0.365	0.295	0.08	0.2555	0.097	0.043	0.1	7
20	1	0.45	0.32	0.1	0.381	0.1705	0.075	0.115	9
21	1	0.355	0.28	0.095	0.2455	0.0955	0.062	0.075	11
22	0	0.38	0.275	0.1	0.2255	0.08	0.049	0.085	10
23	-1	0.565	0.44	0.155	0.9395	0.4275	0.214	0.27	12
24	-1	0.55	0.415	0.135	0.7635	0.318	0.21	0.2	9
25	-1	0.615	0.48	0.165	1.1615	0.513	0.301	0.305	10
26	-1	0.56	0.44	0.14	0.9285	0.3825	0.188	0.3	11
27	-1	0.58	0.45	0.185	0.9955	0.3945	0.272	0.285	11
28	1	0.59	0.445	0.14	0.931	0.356	0.234	0.28	12
29	1	0.605	0.475	0.18	0.9365	0.394	0.219	0.295	15

可以看到，数据集是多维的，所以我们很难画出它的分布情况。每个维度数据的代表的含义没有给出，不过没有关系，我们只要知道最后一列的数据是 y 值就可以了，最后一列代表的是鲍鱼的真实年龄，前面几列的数据是一些鲍鱼的特征，例如鲍鱼壳的层数等。我们不做数据清理，直接用上所有特征，测试下我们的局部加权回归。

新建 `abalone.py` 文件，添加 `rssError` 函数，用于评价最后回归结果。编写代

码如下:

```
# -*- coding:utf-8 -*-
from matplotlib.font_manager import FontProperties
import matplotlib.pyplot as plt
import numpy as np

def loadDataSet(fileName):
    """
    函数说明:加载数据
    Parameters:
        fileName - 文件名
    Returns:
        xArr - x 数据集
        yArr - y 数据集
    """
    numFeat = len(open(fileName).readline().split('\t')) - 1
    xArr = []; yArr = []
    fr = open(fileName)
    for line in fr.readlines():
        lineArr = []
        curLine = line.strip().split('\t')
        for i in range(numFeat):
            lineArr.append(float(curLine[i]))
        xArr.append(lineArr)
        yArr.append(float(curLine[-1]))
    return xArr, yArr

def lwlr(testPoint, xArr, yArr, k = 1.0):
    """
    函数说明:使用局部加权线性回归计算回归系数 w
    Parameters:
        testPoint - 测试样本点
        xArr - x 数据集
        yArr - y 数据集
        k - 高斯核的 k, 自定义参数
    Returns:
        ws - 回归系数
    """
    xMat = np.mat(xArr); yMat = np.mat(yArr).T
    m = np.shape(xMat)[0]
    weights = np.mat(np.eye((m))) # 创建权重对角矩
    # 遍历数据集计算
    for j in range(m):
        每个样本的权重
```

```

        diffMat = testPoint - xMat[j, :]
        weights[j, j] = np.exp(diffMat * diffMat.T/(-2.0 * k**2))
    xTx = xMat.T * (weights * xMat)
    if np.linalg.det(xTx) == 0.0:
        print("矩阵为奇异矩阵,不能求逆")
        return
    ws = xTx.I * (xMat.T * (weights * yMat)) # 计算回归
    系数
    return testPoint * ws

def lwlrTest(testArr, xArr, yArr, k=1.0):
    """
    函数说明:局部加权线性回归测试
    Parameters:
        testArr - 测试数据集,测试集
        xArr - x 数据集,训练集
        yArr - y 数据集,训练集
        k - 高斯核的k,自定义参数
    Returns:
        ws - 回归系数
    """
    m = np.shape(testArr)[0] # 计算测试数据集大小
    yHat = np.zeros(m)
    for i in range(m): # 对每个样本点进行预测
        yHat[i] = lwlr(testArr[i],xArr,yArr,k)
    return yHat

def standRegres(xArr,yArr):
    """
    函数说明:计算回归系数w
    Parameters:
        xArr - x 数据集
        yArr - y 数据集
    Returns:
        ws - 回归系数
    """
    xMat = np.mat(xArr); yMat = np.mat(yArr).T
    xTx = xMat.T * xMat # 根据文中推导的公示计算回归系数
    if np.linalg.det(xTx) == 0.0:
        print("矩阵为奇异矩阵,不能求逆")
        return
    ws = xTx.I * (xMat.T*yMat)
    return ws

```

```

def rssError(yArr, yHatArr):
    """
    误差大小评价函数
    Parameters:
        yArr - 真实数据
        yHatArr - 预测数据
    Returns:
        误差大小
    """
    return ((yArr - yHatArr) **2).sum()

if __name__ == '__main__':
    abX, abY = loadDataSet('abalone.txt')
    print('训练集与测试集相同:局部加权线性回归,核 k 的大小对预测的影响:')
    yHat01 = lwlrTest(abX[0:99], abX[0:99], abY[0:99], 0.1)
    yHat1 = lwlrTest(abX[0:99], abX[0:99], abY[0:99], 1)
    yHat10 = lwlrTest(abX[0:99], abX[0:99], abY[0:99], 10)
    print('k=0.1 时,误差大小为:',rssError(abY[0:99], yHat01.T))
    print('k=1 时,误差大小为:',rssError(abY[0:99], yHat1.T))
    print('k=10 时,误差大小为:',rssError(abY[0:99], yHat10.T))

    print('')

    print('训练集与测试集不同:局部加权线性回归,核 k 的大小是越小越好吗? 更换数据
    集,测试结果如下:')
    yHat01 = lwlrTest(abX[100:199], abX[0:99], abY[0:99], 0.1)
    yHat1 = lwlrTest(abX[100:199], abX[0:99], abY[0:99], 1)
    yHat10 = lwlrTest(abX[100:199], abX[0:99], abY[0:99], 10)
    print('k=0.1 时,误差大小为:',rssError(abY[100:199], yHat01.T))
    print('k=1 时,误差大小为:',rssError(abY[100:199], yHat1.T))
    print('k=10 时,误差大小为:',rssError(abY[100:199], yHat10.T))

    print('')

    print('训练集与测试集不同:简单的线性回归与 k=1 时的局部加权线性回归对比:')
    print('k=1 时,误差大小为:', rssError(abY[100:199], yHat1.T))
    ws = standRegres(abX[0:99], abY[0:99])
    yHat = np.mat(abX[100:199]) * ws
    print('简单的线性回归误差大小:', rssError(abY[100:199], yHat.T.A))

    运行结果如下:

```

```
运行: regression x abalone x
D:\ProgramData\Anaconda3\python.exe E:/机器学习/机器学习实验指导书/代码/P8-I
训练集与测试集相同:局部加权线性回归,核k的大小对预测的影响:
k=0.1时,误差大小为: 56.79663263603432
k=1 时,误差大小为: 429.8905618701754
k=10 时,误差大小为: 549.1181708826999

训练集与测试集不同:局部加权线性回归,核k的大小是越小越好吗? 更换数据集,测试结果如下:
矩阵为奇异矩阵,不能求逆
矩阵为奇异矩阵,不能求逆
矩阵为奇异矩阵,不能求逆
k=0.1时,误差大小为: nan
k=1 时,误差大小为: 573.5261441897916
k=10 时,误差大小为: 517.5711905381474

训练集与测试集不同:简单的线性回归与k=1时的局部加权线性回归对比:
k=1时,误差大小为: 573.5261441897916
简单的线性回归误差大小: 518.636315324306

进程已结束,退出代码0
```

可以看到,当 $k=0.1$ 时,训练集误差小,但是应用于新的数据集之后,误差反而无法计算了。这就是经常说过的过拟合现象。我们训练的模型,我们要保证测试集准确率高,这样训练出的模型才可以应用于新的数据,也就是要加强模型的普适性。可以看到,当 $k=1$ 时,局部加权线性回归和简单的线性回归得到的效果差不多。这也表明一点,必须在未知数据上比较效果才能选取到最佳模型。那么最佳的核大小是 10 吗? 或许是,但如果想得到更好的效果,应该用 10 个不同的样本集做 10 次测试来比较结果。

本次实验练习了如何使用局部加权线性回归来构建模型,可以得到比普通线性回归更好的效果。局部加权线性回归的问题在于,每次必须在整个数据集上运行。也就是说为了做出预测,必须保存所有的训练数据。

（二）银行房屋贷款判断（12 页-28 页）

1、银行房屋贷款申请，银行房屋贷款数据如下。

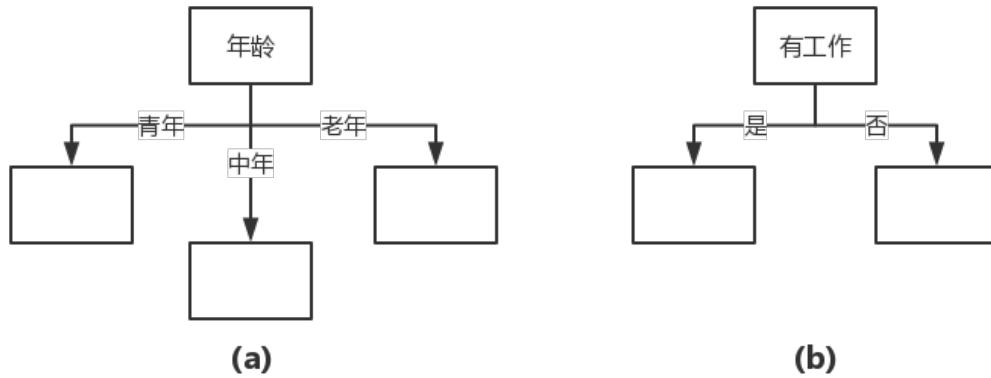
ID	年龄	有工作	有自己的房子	信贷情况	类别(是否个给贷款)
1	青年	否	否	一般	否
2	青年	否	否	好	否
3	青年	是	否	好	是
4	青年	是	是	一般	是
5	青年	否	否	一般	否
6	中年	否	否	一般	否
7	中年	否	否	好	否
8	中年	是	是	好	是
9	中年	否	是	非常好	是
10	中年	否	是	非常好	是
11	老年	否	是	非常好	是
12	老年	否	是	好	是
13	老年	是	否	好	是
14	老年	是	否	非常好	是
15	老年	否	否	一般	否

要求：

- ①构建决策树。
- ②保存和读取决策树。
- ③使用决策树进行分类预测。

我们希望通过所给的训练数据学习一个贷款申请的决策树，用以对未来的贷款申请进行分类，即当新的客户提出贷款申请时，根据申请人的特征利用决策树决定是否批准贷款申请。

特征选择就是决定用哪个特征来划分特征空间。比如，我们通过上述数据表得到两个可能的决策树，分别由两个不同特征的根结点构成。



图(a)所示的根结点的特征是年龄,有3个取值,对应于不同的取值有不同的子结点。图(b)所示的根结点的特征是工作,有2个取值,对应于不同的取值有不同的子结点。两个决策树都可以从此延续下去。问题是:究竟选择哪个特征更好些?这就要求确定选择特征的准则。而信息增益就能够很好地表示这一准则。

什么是信息增益呢?在划分数据集之前之后信息发生的变化成为信息增益,知道如何计算信息增益,我们就可以计算每个特征值划分数据集获得的信息增益,获得信息增益最高的特征就是最好的选择。

(1) 香农熵

在可以评测哪个数据划分方式是最好的数据划分之前,我们必须学习如何计算信息增益。集合信息的度量方式成为香农熵或者简称为熵(entropy)或信息熵,这个名字来源于信息论之父克劳德·香农。

熵定义为信息的期望值。在信息论与概率统计中,熵是表示随机变量不确定性的度量。如果待分类的事务可能划分在多个分类之中,则符号 x_i 的信息定义为

$$l(x_i) = -\log_2 p(x_i)$$

其中 $p(x_i)$ 是选择该分类的概率。上述式中的对数以2为底,也可以e为底(自然对数)。

通过上式,我们可以得到所有类别的信息。为了计算熵,我们需要计算所有类别所有可能值包含的信息期望值(数学期望),通过下面的公式得到

$$H = -\sum_{i=1}^n p(x_i) \log_2 p(x_i)$$

期中 n 是分类的数目。熵越大,随机变量的不确定性就越大。

我们定义贷款申请样本数据表中的数据为训练数据集 D ,则训练数据集 D 的经验熵为 $H(D)$, $|D|$ 表示其样本容量,及样本个数。设有 k 个类 $C_k, k=1,2,3,\dots,k$, $|C_k|$ 为属于类 C_k 的样本个数,熵公式可以写为

$$H(D) = - \sum_{k=1}^k \frac{|C_k|}{|D|} \log_2 \frac{|C_k|}{|D|}$$

根据此公式计算经验熵 $H(D)$ ，分析贷款申请样本数据表中的数据。最终分类结果只有两类，即放贷和不放贷。根据表中的数据统计可知，在 15 个数据中，9 个数据的结果为放贷，6 个数据的结果为不放贷。所以数据集 D 的经验熵 $H(D)$ 为：

$$H(D) = -\frac{9}{15} \log_2 \frac{9}{15} - \frac{6}{15} \log_2 \frac{6}{15} = 0.971$$

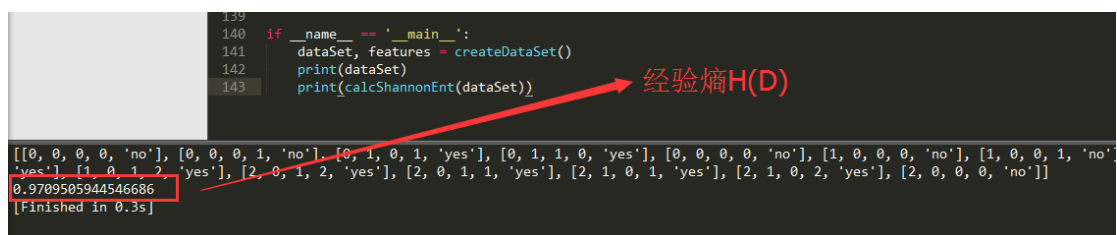
经过计算可知，数据集 D 的经验熵 $H(D)$ 的值为 0.971。

(2) 编写代码计算经验熵

在编写代码之前，我们先对数据集进行属性标注。

- 年龄：0 代表青年，1 代表中年，2 代表老年；
- 有工作：0 代表否，1 代表是；
- 有自己的房子：0 代表否，1 代表是；
- 信贷情况：0 代表一般，1 代表好，2 代表非常好；
- 类别(是否给贷款)：no 代表否，yes 代表是。

确定这些之后，我们就可以创建数据集，并计算熵，代码运行结果如下图所示，代码是先打印训练数据集，然后打印计算的经验熵 $H(D)$ ，程序计算的结果与我们统计计算的结果是一致的，程序没有问题。



```

139
140 if __name__ == '__main__':
141     dataSet, features = createDataSet()
142     print(dataSet)
143     print(calcShannonEnt(dataSet))

```

经验熵H(D)

```

[[0, 0, 0, 0, 'no'], [0, 0, 0, 1, 'no'], [0, 1, 0, 1, 'yes'], [0, 1, 1, 0, 'yes'], [0, 0, 0, 0, 'no'], [1, 0, 0, 0, 'no'], [1, 0, 0, 1, 'no'],
['yes'], [1, 0, 1, 2, 'yes'], [2, 0, 1, 2, 'yes'], [2, 0, 1, 1, 'yes'], [2, 1, 0, 1, 'yes'], [2, 1, 0, 2, 'yes'], [2, 0, 0, 0, 'no']]
0.9709505944546686
[Finished in 0.3s]

```

(3) 信息增益

在上面，我们已经说过，如何选择特征，需要看信息增益。也就是说，信息增益是相对于特征而言的，信息增益越大，特征对最终的分类结果影响也就越大，我们就应该选择对最终分类结果影响最大的那个特征作为我们的分类特征。

在讲解信息增益定义之前，我们还需要明确一个概念，条件熵。

条件熵 $H(Y|X)$ 表示在已知随机变量 X 的条件下随机变量 Y 的不确定性，随机变量 X 给定的条件下随机变量 Y 的条件熵(conditional entropy) $H(Y|X)$ ，定义 X 给定条件下 Y 的条件概率分布的熵对 X 的数学期望为

$$H(Y|X) = \sum_{i=1}^n p_i \bullet H(Y|X=x_i)$$

这里， $p_i = P(X=x_i), i=1,2,\dots,n$

信息增益是相对于特征而言的。所以，特征 A 对训练数据集 D 的信息增益 $G(D,A)$ ，定义为集合 D 的经验熵 $H(D)$ 与特征 A 给定条件下 D 的经验条件熵 $H(D|A)$ 之差，即

$$g(D, A) = H(D) - H(D|A)$$

一般地，熵 $H(D)$ 与条件熵 $H(D|A)$ 之差成为互信息(mutual information)。决策树学习中的信息增益等价于训练数据集中类与特征的互信息。

设特征 A 有 n 个不同的取值 $\{a_1, a_2, \dots, a_n\}$ ，根据特征 A 的取值将 D 划分为 n 个子集 $D_1, D_2, \dots, D_n$ ， $|D_i|$ 为 D_i 的样本个数。记子集 D_i 中属于 C_k 的样本的集合为 D_{ik} ，即 $D_{ik} = D_i \cap C_k$ ， $|D_{ik}|$ 为 D_{ik} 的样本个数。于是条件熵的公式可以写为

$$H(D|A) = \sum_{i=1}^n \frac{|D_i|}{|D|} H(D_i) = - \sum_{i=1}^n \frac{|D_i|}{|D|} \sum_{k=1}^K \frac{|D_{ik}|}{|D_i|} \log \frac{|D_{ik}|}{|D_i|}$$

以贷款申请样本数据表为例进行说明。看下年龄这一列的数据，也就是特征 A_1 ，一共有三个类别，分别是：青年、中年和老年。我们只看年龄是青年的数据，年龄是青年的数据一共有 5 个，所以年龄是青年的数据在训练数据集出现的概率是十五分之五，也就是三分之一。同理，年龄是中年的数据在训练数据集出现的概率也都是三分之一。现在我们只看年龄是青年的数据的最终得到贷款的概率为五分之二，因为在五个数据中，只有两个数据显示拿到了最终的贷款，同理，年龄是中年的数据最终得到贷款的概率分别为五分之三、五分之四。所以计算年龄的信息增益，过程如下：

$$\begin{aligned} g(D, A_1) &= H(D) - [\frac{5}{15} H(D_1) + \frac{5}{15} H(D_2) + \frac{5}{15} H(D_3)] \\ &= 0.971 - [\frac{5}{15} (-\frac{2}{5} \log_2 \frac{2}{5} - \frac{3}{5} \log_2 \frac{3}{5}) + \frac{5}{15} (-\frac{3}{5} \log_2 \frac{3}{5} - \frac{2}{5} \log_2 \frac{2}{5}) + \frac{5}{15} (-\frac{4}{5} \log_2 \frac{4}{5} - \frac{1}{5} \log_2 \frac{1}{5})] \\ &= 0.971 - 0.888 \\ &= 0.083 \end{aligned}$$

同理，计算其余特征的信息增益 $g(D, A_2)$ 、 $g(D, A_3)$ 和 $g(D, A_4)$ 。分别为：

$$\begin{aligned}
g(D, A_2) &= H(D) - \left[\frac{5}{15} H(D_1) + \frac{10}{15} H(D_2) \right] \\
&= 0.971 - \left[\frac{5}{15} \times 0 + \frac{10}{15} \left(-\frac{4}{10} \log_2 \frac{4}{10} - \frac{6}{10} \log_2 \frac{6}{10} \right) \right] \\
&= 0.971 - 0.647 \\
&= 0.324
\end{aligned}$$

$$\begin{aligned}
g(D, A_3) &= H(D) - \left[\frac{6}{15} H(D_1) + \frac{9}{15} H(D_2) \right] \\
&= 0.971 - \left[\frac{6}{15} \times 0 + \frac{9}{15} \left(-\frac{3}{9} \log_2 \frac{3}{9} - \frac{6}{9} \log_2 \frac{6}{9} \right) \right] \\
&= 0.971 - 0.551 \\
&= 0.420
\end{aligned}$$

$$\begin{aligned}
g(D, A_4) &= 0.971 - 0.608 \\
&= 0.363
\end{aligned}$$

最后，比较特征的信息增益，由于特征 A_3 (有自己的房子)的信息增益值最大，所以选择 A_3 作为最优特征。

(4) 计算信息增益

我们已经学会了通过公式计算信息增益，接下来编写代码，计算信息增益，选择最优特征。

后面程序中，`splitDataSet` 函数是用来选择各个特征的子集的，比如选择年龄(第 0 个特征)的青年(用 0 代表)的自己，我们可以调用 `splitDataSet(dataSet, 0, 0)` 这样返回的子集就是年龄为青年的 5 个数据集。`chooseBestFeatureToSplit` 是选择选择最优特征的函数。运行代码结果如下：

```

112     print("第%d个特征的增益为%.3f" % (i, infoGain))      #打印每个特征的信息增益
113     if (infoGain > bestInfoGain):                          #计算信息增益
114         bestInfoGain = infoGain                            #更新信息增益，找到最大的信息增益
115         bestFeature = i                                    #记录信息增益最大的特征的索引值
116     return bestFeature                                     #返回信息增益最大的特征的索引值
117
118 if __name__ == '__main__':
119     dataSet, features = createDataSet()
120     print("最优特征索引值:" + str(chooseBestFeatureToSplit(dataSet)))

```

第0个特征的增益为0.083
 第1个特征的增益为0.324
 第2个特征的增益为0.420
 第3个特征的增益为0.363
 最优特征索引值:2
 [Finished in 0.2s]

对比我们自己计算的结果，发现结果完全正确！最优特征的索引值为 2，也就是特征 A_3 (有自己的房子)。

对 D_2 则需要从特征 A_1 (年龄), A_2 (有工作)和 A_4 (信贷情况)中选择新的特征，计算各个特征的信息增益：

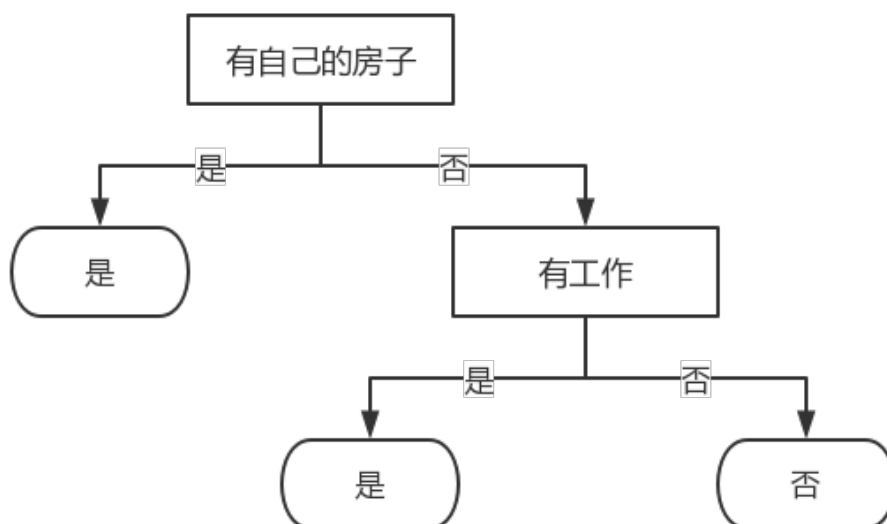
$$g(D_2, A_1) = H(D_2) - H(D_2 | A_1) = 0.251$$

$$g(D_2, A_2) = H(D_2) - H(D_2 | A_2) = 0.918$$

$$g(D_2, A_3) = H(D_2) - H(D_2 | A_3) = 0.474$$

根据计算，选择信息增益最大的特征 A_2 (有工作)作为结点的特征。由于 A_2 有两个可能取值，从这一结点引出两个子结点：一个对应“是”(有工作)的子结点，包含 3 个样本，它们属于同一类，所以这是一个叶结点，类标记为“是”；另一个是对应“否”(无工作)的子结点，包含 6 个样本，它们也属于同一类，所以这也是一个叶结点，类标记为“否”。

这样就生成了一个决策树，该决策树只用了两个特征(有两个内部结点)，生成的决策树如下图所示。



我们使用 ID3 算法，通过计算构建出决策树，接下来，让我们看看如何进行代实现。

使用字典存储决策树的结构，比如上小节我们分析出来的决策树，用字典可以表示为：

```
{'有自己的房子': {0: {'有工作': {0: 'no', 1: 'yes'}}, 1: 'yes'}}
```

创建函数 `majorityCnt` 统计 `classList` 中出现此处最多的元素(类标签)，创建函数 `createTree` 用来递归构建决策树。递归创建决策树时，递归有两个终止条件：第一个停止条件是所有的类标签完全相同，则直接返回该类标签；第二个停止条件是使用完了所有特征，仍然不能将数据划分仅包含唯一类别的分组，即决策树构建失败，特征不够用。此时说明数据纬度不够，由于第二个停止条件无法简单地返回唯一的类标签，这里挑选出现数量最多的类别作为返回值。

运行代码，我们可以看到如下结果：

```
404
405 if __name__ == '__main__':
406     dataSet, labels = createDataSet()
407     featLabels = []
408     myTree = createTree(dataSet, labels, featLabels)
409     print(myTree)

{'有自己的房子': {0: {'有工作': {0: 'no', 1: 'yes'}}, 1: 'yes'}}
[Finished in 3.2s]
```

可见，我们的决策树已经构建完成了。后面我们可以使用强大的 Matplotlib 绘制决策树。

(5) 决策树可视化

这里代码都是关于 Matplotlib 的，如果对于 Matplotlib 不了解的，可以先学习下，Matplotlib 的内容这里就不再累述。可视化需要用到的函数：

getNumLeafs: 获取决策树叶子结点的数目

getTreeDepth: 获取决策树的层数

plotNode: 绘制结点

plotMidText: 标注有向边属性值

plotTree: 绘制决策树

createPlot: 创建绘制面板

我对可视化决策树的程序进行了详细的注释，直接看代码，调试查看即可。

为了显示中文，需要设置 FontProperties，代码编写如下：

```
# -*- coding: UTF-8 -*-
```

```
from matplotlib.font_manager import FontProperties
```

```
import matplotlib.pyplot as plt
```

```
from math import log
```

```
import operator
```

```
import pickle
```

```
"""
```

函数说明: 计算给定数据集的经验熵(香农熵)

Parameters:

dataSet - 数据集

Returns:

shannonEnt - 经验熵(香农熵)

```
"""
```

```
def calcShannonEnt(dataSet):
```

```
    numEntires = len(dataSet)
```

```
    labelCounts = {}
```

```
    #返回数据集的行数
```

```
    #保存每个标签(Label)出现次数
```

的字典

```
for featVec in dataSet:                                #对每组特征向量进行统计
    currentLabel = featVec[-1]                          #提取标签(Label)信息
    if currentLabel not in labelCounts.keys():          #如果标签(Label)没有放入统计次
数字典,添加进去
        labelCounts[currentLabel] = 0
        labelCounts[currentLabel] += 1                #Label 计数
    shannonEnt = 0.0                                    #经验熵(香农熵)
    for key in labelCounts:                             #计算香农熵
        prob = float(labelCounts[key]) / numEntires    #选择该标签(Label)的概率
        shannonEnt -= prob * log(prob, 2)              #利用公式计算
    return shannonEnt                                    #返回经验熵(香农熵)
```

"""

函数说明:创建测试数据集

Parameters:

无

Returns:

dataSet - 数据集

labels - 特征标签

"""

```
def createDataSet():
    dataSet = [[0, 0, 0, 0, 'no'],                    #数据集
               [0, 0, 0, 1, 'no'],
               [0, 1, 0, 1, 'yes'],
               [0, 1, 1, 0, 'yes'],
               [0, 0, 0, 0, 'no'],
               [1, 0, 0, 0, 'no'],
               [1, 0, 0, 1, 'no'],
               [1, 1, 1, 1, 'yes'],
               [1, 0, 1, 2, 'yes'],
               [1, 0, 1, 2, 'yes'],
               [2, 0, 1, 2, 'yes'],
               [2, 0, 1, 1, 'yes'],
               [2, 1, 0, 1, 'yes'],
               [2, 1, 0, 2, 'yes'],
               [2, 0, 0, 0, 'no']]
    labels = ['age', 'job', 'house', 'credit']         #特征标签['年龄', '有工作', '有自己的房子', '信贷情况']
    return dataSet, labels                             #返回数据集和分类属性
```

"""

函数说明:按照给定特征划分数据集

Parameters:

dataSet - 待划分的数据集
axis - 划分数据集的特征
value - 需要返回的特征的值

Returns:

无

"""

```
def splitDataSet(dataSet, axis, value):                #三个输入参数：待划分的数据
集、划分数据集的特征、需要返回的特征值
    retDataSet = []                                    #创建返回的数据集列表（创建新列
表）
    for featVec in dataSet:                            #遍历数据集
        if featVec[axis] == value:
            reducedFeatVec = featVec[:axis]             #去掉 axis 特征
            reducedFeatVec.extend(featVec[axis+1:])     #将符合条件的添加到返回的数据集
            retDataSet.append(reducedFeatVec)
    return retDataSet                                  #返回划分后的数据集
```

"""

函数说明:选择最优特征

Parameters:

dataSet - 数据集

Returns:

bestFeature - 信息增益最大的(最优)特征的索引值

"""

```
def chooseBestFeatureToSplit(dataSet):                #函数含义：选取特征，划分数据
集，计算得出最好的划分数据集的特征
    numFeatures = len(dataSet[0]) - 1                #特征数量
    baseEntropy = calcShannonEnt(dataSet)             #计算数据集的香农熵
    bestInfoGain = 0.0                                #信息增益
    bestFeature = -1                                   #最优特征的索引值
    for i in range(numFeatures):                       #遍历所有特征
        #获取 dataSet 的第 i 个所有特征
        featList = [example[i] for example in dataSet] #将数据集证所有第 i 个特征值或者所
有可存在的值写入新的 list 中
        uniqueVals = set(featList)                   #创建 set 集合 {}, 元素不可重复
        newEntropy = 0.0                             #经验条件熵
        for value in uniqueVals:                       #计算信息增益
            subDataSet = splitDataSet(dataSet, i, value) #subDataSet 划分后的子集
```

```

        prob = len(subDataSet) / float(len(dataSet))          #计算子集的概率
        newEntropy += prob * calcShannonEnt(subDataSet)      #根据公式计算经验条件熵
        infoGain = baseEntropy - newEntropy                  #信息增益
        # print("第%d 个特征的增益为%.3f" % (i, infoGain))    #打印每个特征的信息增
益
        if (infoGain > bestInfoGain):                          #计算信息增益
            bestInfoGain = infoGain                            #更新信息增益，找到最大的信
息增益
            bestFeature = i                                    #记录信息增益最大的特征的索引
值
        return bestFeature                                    #返回信息增益最大的特征的
索引值

```

函数说明:统计 classList 中出现此处最多的元素(类标签)，用于定义叶子节点

Parameters:

classList - 类标签列表

Returns:

sortedClassCount[0][0] - 出现此处最多的元素(类标签)

```

def majorityCnt(classList):
    classCount = {}                                           #保存每个类标签(class)
    出现次数的字典
    for vote in classList:                                    #统计 classList 中每个元素出现的次
数
        if vote not in classCount.keys():classCount[vote] = 0
        classCount[vote] += 1
    sortedClassCount = sorted(classCount.items(), key = operator.itemgetter(1), reverse = True)
    #根据字典的值降序排序
    return sortedClassCount[0][0]                             #返回 classList 中出现次数最多的
元素

```

函数说明:创建决策树

Parameters:

dataSet - 训练数据集

labels - 分类属性标签

featLabels - 存储选择的最优特征标签

Returns:

myTree - 决策树

```

"""
def createTree(dataSet, labels, featLabels):
    classList = [example[-1] for example in dataSet]          #取分类标签(是否放贷:yes or
no)
    if classList.count(classList[0]) == len(classList):        #如果类别完全相同则停止继续
划分
        return classList[0]
    if len(dataSet[0]) == 1 or len(labels) == 0:               #遍历完所有特征
时返回出现次数最多的类标签
        return majorityCnt(classList)
    bestFeat = chooseBestFeatureToSplit(dataSet)               #选择最优特征
    bestFeatLabel = labels[bestFeat]                           #最优特征的标签
    featLabels.append(bestFeatLabel)                           #创建列表
    myTree = {bestFeatLabel: {}}                               #根据最优特征的标签生成
树
    del(labels[bestFeat])                                       #删除已经使用特征标签
    featValues = [example[bestFeat] for example in dataSet]    #得到训练集中所有最优特
征的属性值
    uniqueVals = set(featValues)                                #去掉重复的属性值
    for value in uniqueVals:                                    #遍历特征，创建决策树。
        subLabels = labels[:]                                  #复制类标签，存在
subLabels 中，保证每次调用不改变原始列表内容
        myTree[bestFeatLabel][value] = createTree(splitDataSet(dataSet, bestFeat, value),
subLabels, featLabels)

    return myTree
"""

```

函数说明:获取决策树叶子结点的数目

Parameters:

myTree - 决策树

Returns:

numLeafs - 决策树的叶子结点的数目

```

"""
def getNumLeafs(myTree):
    numLeafs = 0                                               #初始化叶子
    firstStr = next(iter(myTree))                             #python3 中 myTree.keys() 返回的
是 dict_keys, 不在是 list, 所以不能使用 myTree.keys()[0] 的方法获取结点属性, 可以使用
list(myTree.keys())[0]
    secondDict = myTree[firstStr]                             #获取下一组字典
    for key in secondDict.keys():

```

```

        if type(secondDict[key]).__name__=='dict':           #测试该结点是否为字典，如
            果不是字典，代表此结点为叶子结点，如果是字典，则该节点是判断节点，继续循环
            numLeafs += getNumLeafs(secondDict[key])
        else:    numLeafs +=1
    return numLeafs

```

"""

函数说明:获取决策树的层数

Parameters:

myTree - 决策树

Returns:

maxDepth - 决策树的层数

"""

```

def getTreeDepth(myTree):
    maxDepth = 0           #初始化决策树深度
    firstStr = next(iter(myTree))           #python3 中 myTree.keys() 返回的
    是 dict_keys, 不在是 list, 所以不能使用 myTree.keys()[0] 的方法获取结点属性, 可以使用
    list(myTree.keys())[0]
    secondDict = myTree[firstStr]           #获取下一个字典
    for key in secondDict.keys():
        if type(secondDict[key]).__name__=='dict':           #测试该结点是否为字典，如
            果不是字典，代表此结点为叶子结点
            thisDepth = 1 + getTreeDepth(secondDict[key])
        else:    thisDepth = 1
        if thisDepth > maxDepth: maxDepth = thisDepth           #更新层数
    return maxDepth

```

"""

函数说明:绘制结点

Parameters:

nodeTxt - 结点名

centerPt - 文本位置

parentPt - 标注的箭头位置

nodeType - 结点格式

Returns:

无

"""

```

def plotNode(nodeTxt, centerPt, parentPt, nodeType):
    arrow_args = dict(arrowstyle="<-")           #定义箭头格式
    font = FontProperties(fname=r"c:\windows\fonts\simnsunb.ttf", size=14)           #设置中文字体

```

```

createPlot.ax1.annotate(nodeTxt, xy=parentPt, xycoords='axes fraction', #绘制结点
    xytext=centerPt, textcoords='axes fraction',
    va="center", ha="center", bbox=nodeType, arrowprops=arrow_args, fontproperties=font)

"""
函数说明:标注有向边属性值

Parameters:
    cntrPt、parentPt - 用于计算标注位置
    txtString - 标注的内容
Returns:
    无

"""

def plotMidText(cntrPt, parentPt, txtString):
    xMid = (parentPt[0]-cntrPt[0])/2.0 + cntrPt[0] #计算标注位置
    yMid = (parentPt[1]-cntrPt[1])/2.0 + cntrPt[1]
    createPlot.ax1.text(xMid, yMid, txtString, va="center", ha="center", rotation=30)

"""
函数说明:绘制决策树

Parameters:
    myTree - 决策树(字典)
    parentPt - 标注的内容
    nodeTxt - 结点名
Returns:
    无

"""

def plotTree(myTree, parentPt, nodeTxt):
    decisionNode = dict(boxstyle="sawtooth", fc="0.8") #设置
    # 结点格式(文本框)
    leafNode = dict(boxstyle="round4", fc="0.8") #设置
    # 叶结点格式(箭头)
    numLeafs = getNumLeafs(myTree) #获取
    # 取决策树叶结点数目, 决定了树的宽度
    depth = getTreeDepth(myTree) #获取
    # 决策树层数
    firstStr = next(iter(myTree)) #下个字典
    cntrPt = (plotTree.xOff + (1.0 + float(numLeafs))/2.0/plotTree.totalW, plotTree.yOff) #中心
    # 位置
    plotMidText(cntrPt, parentPt, nodeTxt) #标注有

```

向边属性值

```
plotNode(firstStr, cntrPt, parentPt, decisionNode) #绘制结
点
secondDict = myTree[firstStr] #下一个
字典，也就是继续绘制子结点
plotTree.yOff = plotTree.yOff - 1.0/plotTree.totalD #y
偏移
for key in secondDict.keys():
    if type(secondDict[key]).__name__=='dict': #测试该
        plotTree(secondDict[key],cntrPt,str(key)) #不是
        叶结点，递归调用继续绘制
    else: #如果是叶
        结点，绘制叶结点，并标注有向边属性值
        plotTree.xOff = plotTree.xOff + 1.0/plotTree.totalW
        plotNode(secondDict[key], (plotTree.xOff, plotTree.yOff), cntrPt, leafNode)
        plotMidText((plotTree.xOff, plotTree.yOff), cntrPt, str(key))
        plotTree.yOff = plotTree.yOff + 1.0/plotTree.totalD
```

"""

函数说明:创建绘制面板

Parameters:

inTree - 决策树(字典)

Returns:

无

"""

```
def createPlot(inTree):
    fig = plt.figure(1, facecolor='white') #创建新图
    形fig
    fig.clf() #清空fig
    绘图区
    axprops = dict(xticks=[], yticks=[])
    #建立坐标系
    createPlot.ax1 = plt.subplot(111, frameon=False, **axprops)
    #去掉x、y轴
    plotTree.totalW = float(getNumLeafs(inTree)) #获
    取决策树叶结点数目（确定x轴长度）
    plotTree.totalD = float(getTreeDepth(inTree)) #获取
    决策树层数（确定y轴高度）
    plotTree.xOff = -0.5/plotTree.totalW; plotTree.yOff = 1.0; #x偏
    移
    plotTree(inTree, (0.5,1.0), "") #绘制
```

决策树

```
plt.show()
```

#显示绘

制结果

```
"""
```

函数说明:使用决策树分类

Parameters:

inputTree - 已经生成的决策树

featLabels - 存储选择的最优特征标签

testVec - 测试数据列表, 顺序对应最优特征标签

Returns:

classLabel - 分类结果

```
"""
```

```
def classify(inputTree, featLabels, testVec):
```

```
    firstStr = next(iter(inputTree))
```

#获取决策

树结点

```
    secondDict = inputTree[firstStr]
```

#下一个

字典

```
    featIndex = featLabels.index(firstStr)
```

```
    for key in secondDict.keys():
```

```
        if testVec[featIndex] == key:
```

```
            if type(secondDict[key]).__name__ == 'dict':
```

```
                classLabel = classify(secondDict[key], featLabels, testVec)
```

```
            else: classLabel = secondDict[key]
```

```
    return classLabel
```

```
"""
```

函数说明:存储决策树

Parameters:

inputTree - 已经生成的决策树

filename - 决策树的存储文件名

Returns:

无

```
"""
```

```
def storeTree(inputTree, filename):
```

```
    with open(filename, 'wb') as fw:
```

```
        pickle.dump(inputTree, fw)
```

```
"""
```

函数说明:读取决策树

Parameters:

filename - 决策树的存储文件名

Returns:

pickle.load(fr) - 决策树字典

```
"""
```

```
def grabTree(filename):
```

```
    fr = open(filename, 'rb')
```

```
    return pickle.load(fr)
```

```
if __name__ == '__main__':
```

```
    dataSet, labels = createDataSet()
```

```
    featLabels = []
```

```
    myTree = createTree(dataSet, labels, featLabels)
```

```
    createPlot(myTree)
```

```
    testVec = [0,1]
```

#测试数据

```
    result = classify(myTree, featLabels, testVec)
```

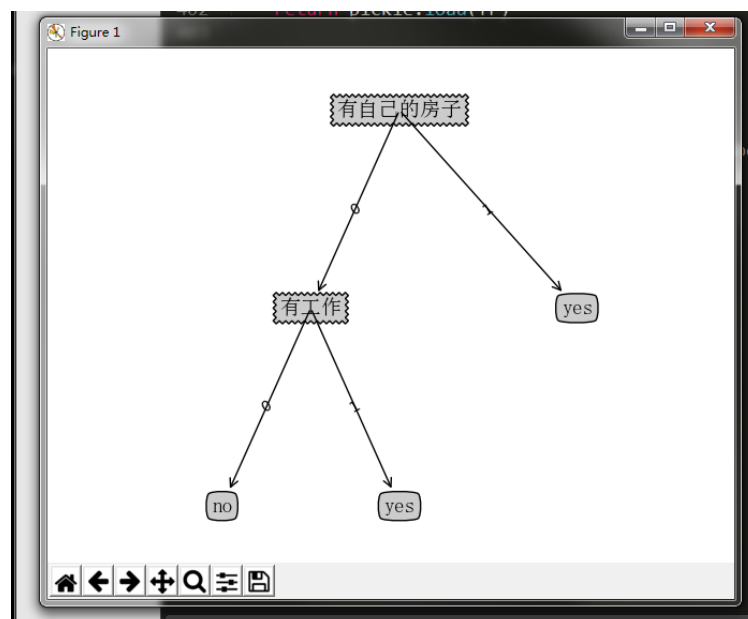
```
    if result == 'yes':
```

```
        print('放贷')
```

```
    if result == 'no':
```

```
        print('不放贷')
```

不出意外的话，我们就可以得到如下结果，可以看到决策树绘制完成。
plotNode 函数的工作就是绘制各个结点，比如有自己的房子、有工作、yes、no，
包括内结点和叶子结点。plotMidText 函数的工作就是绘制各个有向边的属性，
例如各个有向边的 0 和 1。



（6）决策树生成和修剪

我们已经学习了从数据集构造决策树算法所需要的子功能模块，包括熵的计算和最优特征的选择，其工作原理如下：**得到原始数据集，然后基于最好的属性值划分数据集，由于特征值可能多于两个，因此可能存在大于两个分支的数据集划分。第一次划分之后，数据集被向下传递到树的分支的下一个结点。在这个结点上，我们可以再次划分数据。因此我们可以采用递归的原则处理数据集。**

构建决策树的算法有很多，比如 C4.5、ID3 和 CART，这些算法在运行时并不总是在每次划分数据分组时都会消耗特征。由于特征数目并不是每次划分数据分组时都减少，因此这些算法在实际使用时可能引起一定的问题。目前我们并不需要考虑这个问题，只需要在算法开始运行前计算列的数目，查看算法是否使用了所有属性即可。

决策树生成算法递归地产生决策树，直到不能继续下去未为止。这样产生的树往往对训练数据的分类很准确，但对未知的测试数据的分类却没有那么准确，即出现过拟合现象。过拟合的原因在于学习时过多地考虑如何提高对训练数据的正确分类，从而构建出过于复杂的决策树。解决这个问题的办法是考虑决策树的复杂度，对已生成的决策树进行简化。

四：实验要求

- 1、根据实验讲义完成相应实验内容，对比实验结果，调整参数，保证结果的正确性；
- 2、记录实验过程，撰写电子版实验报告，注意每张结果图片上需注明同学们的“学号+姓名”；
- 3、完成电子版实验报告后，调整报告格式内容，不限于字体、字号、标题、段落、图片大小等等，保证报告结果正确且美观，检查无误在学习通平台提交电子版实验报告文件，提交前修改电子版文件名称为“学号+姓名+实验 2”；
- 4、打印纸质版实验报告进行保存，所有实验结束后整体装订成册，交给各班学习委员。